

C++ 并行程序设计

目 录

1 引言：为什么需要 C++ 并行编程	3
2 C++ 基础速览：并发的前置知识	3
2.1 面向对象与封装	3
2.2 RAII：资源获取即初始化	4
2.3 移动语义与 <code>std::move</code>	4
2.4 智能指针	5
2.5 Lambda 表达式	5
2.6 模板与泛型编程	5
2.7 STL 容器与算法	6
3 线程管理： <code>std::thread</code>	6
3.1 创建线程	6
3.2 Most Vexing Parse：最令人苦恼的解析	7
3.3 <code>join()</code> 与 <code>detach()</code>	7
3.4 RAII 线程守卫	8
3.5 传递参数给线程函数	8
3.6 移动语义与线程所有权	9
3.7 线程标识	9
3.8 <code>hardware_concurrency()</code> 与线程数选择	10
4 共享数据与互斥： <code>std::mutex</code>	10
4.1 竞态条件	10
4.2 <code>std::mutex</code> 与 <code>lock_guard</code>	11
4.3 <code>unique_lock</code>	12
4.4 死锁及其破解	12
4.5 互斥量最佳实践	13
5 条件变量： <code>std::condition_variable</code>	14
5.1 轮询的问题	14
5.2 <code>wait</code> 与 <code>notify</code>	14
5.3 虚假唤醒	15
5.4 生产者-消费者模式	15
5.5 线程安全队列	16
6 C++17 并行算法	18
6.1 执行策略	18
6.2 常用并行算法	18
6.3 性能考量	19

7 实战: parallel_accumulate 并行求和	19
8 框架选型: OpenMP vs C++ Threads vs MPI	20
9 本章你将学会	21
10 要点速查	21
11 小结	22

1 引言：为什么需要 C++ 并行编程

现代 CPU 普遍采用多核架构，单线程程序只能利用其中一个核心，相当于买了一台八核电脑却只用了八分之一的算力。要充分利用多核性能，我们必须编写能够同时执行多个任务的程序，这就是**并行程序设计**（Parallel Programming）。

C++11 起在标准库中内建了 `std::thread`、`std::mutex`、`std::condition_variable`、`std::future` 等并发原语，开发者无需依赖 POSIX 线程或第三方库即可编写跨平台的多线程程序。到 C++17，标准库进一步引入了并行算法，只需加一个参数就能让 `std::sort`、`std::for_each` 等算法自动并行执行。

直觉 你可以把多核 CPU 想象成一个工作室里有多个工人（线程）。如果只有一个工人在干活（单线程），其他工人都闲着，效率自然低。并行编程的核心就是：把任务拆分成独立的部分，分给多个工人同时干，最后汇总结果。难点在于：工人之间需要共享工具（数据），如何避免争抢（竞态条件）和互相等待（死锁）。

本讲义将围绕 C++ 并行编程展开，分为三个部分：首先快速回顾 C++ 中与并发密切相关的语言特性（面向对象、RAII、移动语义、Lambda、模板、STL）；然后深入讲解 C++11 并发库的三大支柱：`std::thread`（线程管理）、`std::mutex`（互斥同步）、`std::condition_variable`（条件等待）；最后介绍 C++17 并行算法和框架选型建议。

2 C++ 基础速览：并发的前置知识

在进入并发编程之前，我们需要掌握几个 C++ 语言特性。它们是理解并发库代码的基础：线程函数常用 Lambda 表达式，线程所有权依赖移动语义，资源管理靠 RAII，而线程容器和泛型算法离不开模板与 STL。

2.1 面向对象与封装

C++（C Plus Plus）在 C 的过程式编程基础上引入了**面向对象编程**（Object-Oriented Programming, OOP）。类（Class）将数据和操作数据的函数捆绑在一起，通过**访问说明符**（Access Specifier）控制外部访问权限：

- `public`：任何代码都可以访问
- `private`：只有类自身可以访问
- `protected`：类自身及其派生类可以访问

```
class BankAccount {
private:
    double balance;           // 外部无法直接访问
public:
    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
    double getBalance() const { return balance; } // 只读访问
};
```

封装（Encapsulation）的好处是：内部实现可以随时修改而不影响使用者，同时防止外部代码意外破坏数据。在并发编程中，我们经常把数据和保护它的互斥锁一起封装在类里，这就是“线程安全类”的基本思路。

2.2 RAII：资源获取即初始化

RAII（Resource Acquisition Is Initialization，资源获取即初始化）是 C++ 最核心的设计哲学：在构造函数中获取资源，在析构函数中释放资源。这样无论函数正常返回还是因异常退出，资源都能被正确释放。

直觉 你可以把 RAII 想象成酒店的房卡：入住时领取（构造函数加锁/分配内存），退房时归还（析构函数解锁/释放内存）。即使你匆忙离开忘了归还，酒店也能自动处理。C++ 中变量的作用域结束就等于“退房”，析构函数自动调用。

```
class Point {
public:
    Point() { std::cout << "Constructed" << std::endl; }
    ~Point() { std::cout << "Destroyed" << std::endl; }
};

void func() {
    Point p;           // 构造：获取资源
    // ... 使用 p ...
}                       // 析构：释放资源（即使 func 抛出异常）
```

RAII 在并发编程中至关重要：std::lock_guard 利用 RAII 实现“构造时加锁、析构时解锁”，即使临界区代码抛出异常也能自动解锁，避免死锁。

2.3 移动语义与 std::move

C++11 引入了**移动语义**（Move Semantics），允许将资源（如动态内存、文件句柄）的所有权从一个对象转移到另一个对象，避免昂贵的深拷贝。

直觉 传统的拷贝像是把一本书一字不差地抄一遍，费时费力。移动像是直接把书递过去，原来的位置变空。对于线程、文件句柄等不可复制的资源，移动是唯一的转移方式。

```
class Point {
public:
    // 拷贝构造：深拷贝
    Point(const Point& other) : x(other.x), y(other.y) {}
    // 移动构造：转移所有权，原对象置空
    Point(Point&& other) noexcept : x(other.x), y(other.y) {
        other.x = 0; other.y = 0;
    }
};

Point p1(10, 20);
Point p2 = std::move(p1); // 调用移动构造，p1 变为空状态
```

`std::move` 本身不移动任何东西，它只是一个类型转换，将左值转换为右值引用，从而触发移动构造函数。`std::thread` 是**只能移动**（move-only）的类型，不能拷贝，只能通过 `std::move` 转移所有权。

2.4 智能指针

智能指针（Smart Pointer）是 RAII 在内存管理上的应用。`std::unique_ptr` 独占资源所有权，出作用域自动释放：

```
#include <memory>
```

```
auto ptr = std::make_unique<Resource>(42);    // 自动分配
// ... 使用 ptr ...
// 出作用域自动 delete, 无需手动释放
```

对比 C 的 `malloc/free` 和 C++ 的 `new/delete`，智能指针杜绝了忘记释放内存导致的内存泄漏。在多线程编程中，可以用 `std::move` 将 `unique_ptr` 的所有权转移给线程函数。

2.5 Lambda 表达式

Lambda 表达式（Lambda Expression）是 C++11 引入的匿名函数语法，是编写线程函数最常用的方式：

```
auto add = [](int a, int b) -> int {
    return a + b;
};
std::cout << add(3, 4) << std::endl;    // 输出 7
```

语法结构为 `[捕获](参数) -> 返回类型 { 函数体 }`。其中**捕获子句**（Capture Clause）决定了 Lambda 如何访问外部变量：

- `[=]`：按值捕获所有外部变量
- `[&]`：按引用捕获所有外部变量
- `[id]`：按值捕获指定变量
- `[&id]`：按引用捕获指定变量

在并发编程中，Lambda 是最常用的线程函数形式，因为它可以方便地捕获上下文变量：

```
int id = 1;
std::thread t([id]() {
    std::cout << "Thread " << id << " running" << std::endl;
});
t.join();
```

注意：按引用捕获 `[&]` 在多线程中要格外小心，确保被引用的变量在线程执行期间仍然有效（未被销毁），否则会导致未定义行为。

2.6 模板与泛型编程

模板（Template）允许编写与类型无关的代码，编译器根据实际使用类型自动生成对应版本：

```
template<typename T>
T max_value(T a, T b) {
```

```
    return a > b ? a : b;
}
```

```
int result1 = max_value(10, 20);          // T = int
double result2 = max_value(3.14, 2.71); // T = double
```

STL 的所有容器和算法都是模板，如 `std::vector<int>`、`std::vector<std::thread>`。后面的 `parallel_accumulate` 示例就是一个函数模板。

2.7 STL 容器与算法

标准模板库（Standard Template Library, STL）提供现成的数据结构和算法：

- 容器（Container）：`std::vector`（动态数组）、`std::map`（键值对）、`std::queue`（队列）等
- 算法（Algorithm）：`std::sort`（排序）、`std::find`（查找）、`std::accumulate`（累加）、`std::for_each`（遍历）等
- 迭代器（Iterator）：访问容器元素的统一接口

```
std::vector<int> numbers = {64, 34, 25, 12, 22};
std::sort(numbers.begin(), numbers.end());
int sum = std::accumulate(numbers.begin(), numbers.end(), 0); // 157
```

在并发编程中，`std::vector<std::thread>` 是管理多个线程的常用容器，`std::accumulate` 是并行求和的基础。

3 线程管理：std::thread

`std::thread` 是 C++11 引入的线程类，定义在 `<thread>` 头文件中。构造时传入一个可调用对象（Callable Object），线程便立即开始执行。可调用对象包括：函数指针、函数对象（重载了 `operator()` 的类）、Lambda 表达式、成员函数指针。

3.1 创建线程

最简单的线程创建示例：

```
#include <iostream>
#include <thread>

void hello() {
    std::cout << "Hello from thread!" << std::endl;
}

int main() {
    std::thread t(hello); // 传入函数指针，线程立即启动
    t.join();             // 等待线程完成
    return 0;
}
```

逐行解读：`#include <thread>` 引入线程库；`std::thread t(hello)` 创建线程对象 `t`，将其关联到一个新线程执行 `hello` 函数；`t.join()` 让主线程阻塞等待 `t` 执行完毕。

也可以使用 Lambda 表达式，这在实际开发中更常见：

```
std::thread t([] {
    std::cout << "Hello from lambda thread!" << std::endl;
});
```

```
});
t.join();

    线程函数也可以带参数:

void worker(int id, const std::string& name) {
    std::cout << "Worker " << id << " (" << name << ") running" << std::endl;
}

std::thread t(worker, 42, "Alice");    // 参数直接跟在函数后
t.join();
```

3.2 Most Vexing Parse: 最令人苦恼的解析

当传入函数对象时，会遇到 C++ 中著名的语法陷阱：**Most Vexing Parse**（最令人苦恼的解析）。编译器会将 `std::thread t(background_task());` 解析为函数声明而非变量定义：

```
struct background_task {
    void operator()() { /* do work */ }
};

std::thread t(background_task());    // 错误！被解析为函数声明
```

解决方案是使用**统一初始化**（Uniform Initialization）花括号语法，或使用 Lambda：

```
std::thread t1{background_task()};    // 花括号：统一初始化
std::thread t2((background_task()));    // 额外括号
std::thread t3(background_task{});    // 临时对象语法
std::thread t4([] { /* do work */ });    // 推荐：直接用 Lambda
```

3.3 join() 与 detach()

`std::thread` 对象有两种结束方式：

- **join()**：调用线程阻塞等待目标线程完成。适合“我需要结果才能继续”的场景。
- **detach()**：将线程与对象分离，线程在后台独立运行。适合“启动后不用管”的守护线程。

```
std::thread t1(hello);
t1.join();    // 阻塞等待 t1 完成

std::thread t2(hello);
t2.detach();    // t2 在后台独立运行，t2 对象不再关联任何线程
```

关键规则：线程对象在销毁前必须 `join` 或 `detach`，否则触发 `std::terminate()` 终止程序。可以用 `joinable()` 检查线程是否可以 `join`：

```
std::thread t(worker);
if (t.joinable()) {
    t.join();    // join 后 joinable() 返回 false
}

std::cout << t.joinable() << std::endl;    // 输出 0 (false)
```

detach 后线程句柄将不再有效，无法再 *join*。需确保分离线程访问的资源在其生命周期内有效，否则会导致未定义行为。

3.4 RAII 线程守卫

如果线程创建后、join 前发生了异常，join() 将被跳过，程序崩溃。利用 RAII 可以自动管理这一过程：

```
class thread_guard {
    std::thread& t;
public:
    explicit thread_guard(std::thread& t_) : t(t_) {}
    ~thread_guard() {
        if (t.joinable()) {    // 检查是否可 join
            t.join();          // 自动 join
        }
    }
    thread_guard(const thread_guard&) = delete;    // 禁止拷贝
    thread_guard& operator=(const thread_guard&) = delete;    // 禁止赋值
};

void do_work() {
    std::thread t(some_function);
    thread_guard g(t);    // 用 RAII 守护线程
    do_something_that_might_throw();    // 即使抛出异常
}    // g 析构时自动 join
```

逐行解读：thread_guard 持有线程的引用；析构函数检查 joinable() 后自动 join；= delete 禁止拷贝和赋值，防止线程对象被意外复制。当 do_work 因异常提前退出时，g 的析构函数自动执行 join，避免程序崩溃。

3.5 传递参数给线程函数

线程构造函数默认将参数按值拷贝（copy as rvalue）。若需传引用，必须用 std::ref 包装：

```
void increment(int& x) { x++; }

int counter = 0;
std::thread t(increment, std::ref(counter));    // std::ref 传递引用
t.join();
// counter 现在为 1
```

直觉 线程构造函数会把参数拷贝一份存到内部，这保证了即使原变量被销毁，线程内仍有有效副本。但这也意味着如果不加 std::ref，线程修改的是拷贝而非原变量。

传递成员函数时，需要同时传入对象指针：

```
class X {
public:
    void do_work(int n) {
        std::cout << "Work " << n << " done" << std::endl;
    }
};

X my_x;
```



```
std::thread t(&X::do_work, &my_x, 42); // 成员函数指针 + 对象指针 + 参数
t.join();
```

传字符串字面量时要小心：char[] 到 std::string 的转换可能在新线程中执行，如果原缓冲区已销毁就会出错。解决方法是在传参时显式构造 std::string(buffer)。

3.6 移动语义与线程所有权

std::thread 是**只能移动**（move-only）的类型：不能拷贝，只能移动。这意味着可以将线程所有权从一个变量转移到另一个，但不能有两个变量指向同一个线程：

```
std::thread t1(hello);
// std::thread t2 = t1; // 编译错误：不可拷贝
std::thread t3 = std::move(t1); // OK：移动后 t1 不再关联任何线程
```

这使得 std::vector<std::thread> 成为管理多线程的常用容器：

```
std::vector<std::thread> threads;
for (int i = 0; i < 8; ++i) {
    threads.emplace_back(worker, i); // 直接在线程容器中构造线程
}
for (auto& t : threads) {
    t.join(); // 逐个等待完成
}
```

也可以用 scoped_thread 将 RAII 和移动语义结合，自动管理线程容器：

```
class scoped_thread {
    std::thread t;
public:
    explicit scoped_thread(std::thread t_) : t(std::move(t_)) {
        if (!t.joinable()) throw std::logic_error("No thread");
    }
    ~scoped_thread() { t.join(); }
};

std::vector<scoped_thread> threads;
for (int i = 0; i < 3; ++i) {
    threads.emplace_back(std::thread(worker, i)); // 移动构造
}
// 作用域结束时自动 join 所有线程
```

3.7 线程标识

每个线程有一个唯一的**线程标识符**（Thread ID），通过 std::this_thread::get_id() 获取当前线程 ID，或用 t.get_id() 获取线程对象的 ID：

```
std::cout << "Main thread ID: " << std::this_thread::get_id() << std::endl;
std::thread t(worker);
std::cout << "Worker thread ID: " << t.get_id() << std::endl;
t.join();
std::cout << "After join: " << t.get_id() << std::endl; // 默认 ID
```

线程 ID 可比较、可哈希，能用作容器的键，在调试和线程识别中很有用。

3.8 hardware_concurrency() 与线程数选择

std::thread::hardware_concurrency() 返回硬件支持的并发线程数提示值：

```
unsigned int num_threads = std::thread::hardware_concurrency();
if (num_threads == 0) num_threads = 2;    // 回退值

std::vector<std::thread> threads;
for (unsigned int i = 0; i < num_threads; ++i) {
    threads.emplace_back(worker, i);
}
for (auto& t : threads) t.join();
```

该函数可能返回 0（表示无法确定），使用时需做防御性检查。线程数并非越多越好：线程创建和切换有开销，数据量小时用太多线程反而更慢。

4 共享数据与互斥：std::mutex

多线程编程中最常见的问题就是**竞态条件**（Race Condition）：多个线程同时读写同一变量，且至少一个线程在写，导致结果取决于线程调度的先后顺序。

4.1 竞态条件

以 counter++ 为例，这看似一行的操作实际包含三个步骤：从内存读取值、将值加一、将新值写回内存。对应汇编代码如下：

```
mov eax, [counter]    ; 读取
add eax, 1            ; 加一
mov [counter], eax    ; 写回
```

如果线程 A 在读取和写入之间被抢占，线程 B 也读取了旧值，两个线程都加一后写回，结果只增加了 1 而非 2。这就是**丢失更新**（Lost Update）问题。

例 假设 counter = 0，两个线程各执行一次 counter++：

1. 线程 1 读取 counter = 0
2. 线程 2 读取 counter = 0（线程 1 还没写回）
3. 线程 1 计算 0 + 1 = 1，写回 counter = 1
4. 线程 2 计算 0 + 1 = 1，写回 counter = 1

结果：counter = 1，而不是预期的 2。一次更新丢失了！

如果每个线程执行 100000 次自增，4 个线程预期得到 400000，实际可能只得到 387234。每次运行结果不同，这就是竞态条件的危险之处：bug 不可复现，难以调试。

复现竞态条件的完整代码：

```
#include <iostream>
#include <thread>
#include <vector>

int counter = 0;

void increment() {
```

```

    for (int i = 0; i < 100000; ++i) {
        counter++;           // 非原子操作
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 4; ++i) {
        threads.emplace_back(increment);
    }
    for (auto& t : threads) {
        t.join();
    }
    std::cout << "Expected: 400000, Actual: " << counter << std::endl;
    return 0;
}

```

典型输出: Expected: 400000, Actual: 387234 (每次运行结果不同)。

4.2 std::mutex 与 lock_guard

互斥量 (Mutex) 是最基本的同步原语: 同一时刻只允许一个线程进入**临界区** (Critical Section)。std::lock_guard 是 RAII 风格的互斥锁包装器, 构造时加锁, 析构时解锁, 是最常用且最安全的加锁方式:

```

#include <mutex>

int counter = 0;
std::mutex mtx;

void increment() {
    for (int i = 0; i < 100000; ++i) {
        std::lock_guard<std::mutex> lock(mtx);    // 构造时加锁
        counter++;
    }      // lock 析构时自动解锁
}

```

加锁后输出稳定为 400000。lock_guard 的 RAII 特性保证了即使 counter++ 抛出异常, 锁也能在析构时释放。

关键原则: 把数据和保护它的互斥锁放在一起。可以封装成线程安全类:

```

class ThreadSafeCounter {
private:
    int count = 0;
    mutable std::mutex mtx;    // mutable 允许 const 方法加锁
public:
    void increment() {
        std::lock_guard<std::mutex> lock(mtx);
        ++count;
    }
    int get() const {
        std::lock_guard<std::mutex> lock(mtx);
        return count;
    }
}

```

```

    }
};

```

mutable 关键字允许在 const 成员函数中修改 mtx，因为加锁本身是一种逻辑上的 const 操作。

4.3 unique_lock

std::unique_lock 比 lock_guard 更灵活：支持延迟加锁（defer_lock）、尝试加锁（try_lock）、手动 lock()/unlock()。条件变量必须搭配 unique_lock 使用，因为 wait 需要在等待时释放锁、唤醒时重新获取锁，而 lock_guard 不支持手动解锁。

```

std::unique_lock<std::mutex> lock(mtx);           // 构造时加锁
// 等价于 lock_guard，但可以手动操作
lock.unlock();                                   // 手动解锁
// ... 做一些不需要锁的操作 ...
lock.lock();                                    // 重新加锁

```

延迟加锁示例：

```

std::unique_lock<std::mutex> lock(mtx, std::defer_lock); // 延迟加锁
// ... 准备工作 ...
lock.lock();                                           // 需要时才加锁

```

4.4 死锁及其破解

死锁（Deadlock）是两个或多个线程互相等待对方持有的锁，导致永久阻塞。经典场景：线程 1 先锁 A 再锁 B，线程 2 先锁 B 再锁 A，两个线程同时运行就会死锁。

直觉 两个人面对面走来，都挡住对方的路。A 想往左走，B 也往左走，还是挡住。A 等 B 先动，B 等 A 先动，永远等下去。死锁就是这样：循环等待，无人让步。

```

std::mutex mutex1, mutex2;

void thread1() {
    std::lock_guard<std::mutex> lock1(mutex1); // 先锁 mutex1
    std::lock_guard<std::mutex> lock2(mutex2); // 再锁 mutex2 (可能死锁)
}
void thread2() {
    std::lock_guard<std::mutex> lock2(mutex2); // 先锁 mutex2
    std::lock_guard<std::mutex> lock1(mutex1); // 再锁 mutex1 (可能死锁)
}

```

有三种破解方法。

解法一：统一加锁顺序。约定所有线程以相同顺序获取锁：

```

void thread1() {
    std::lock_guard<std::mutex> l1(mutex1); // 都先锁 mutex1
    std::lock_guard<std::mutex> l2(mutex2); // 再锁 mutex2
}
void thread2() {
    std::lock_guard<std::mutex> l1(mutex1); // 同样顺序
    std::lock_guard<std::mutex> l2(mutex2);
}

```

解法二：std::lock() 原子锁定。C++11 提供 std::lock(), 使用死锁避免算法一次性锁定多个互斥锁：

```
void task_safe() {
    std::unique_lock<std::mutex> l1(mutex1, std::defer_lock);
    std::unique_lock<std::mutex> l2(mutex2, std::defer_lock);
    std::lock(l1, l2);    // 原子地锁定两个互斥锁，不会死锁
}
```

解法三：按地址排序。通过比较互斥锁的地址来决定加锁顺序：

```
void task_by_addr() {
    if (&mutex1 < &mutex2) {
        std::lock_guard<std::mutex> l1(mutex1);
        std::lock_guard<std::mutex> l2(mutex2);
    } else {
        std::lock_guard<std::mutex> l2(mutex2);
        std::lock_guard<std::mutex> l1(mutex1);
    }
}
```

例 银行转账是死锁的经典场景。A 向 B 转账时锁定 A 的账户再锁 B 的账户，同时 B 向 A 转账锁定 B 的账户再锁 A 的账户，就可能死锁。用按地址排序的方法可以解决：

```
class BankAccount {
    double balance;
    mutable std::mutex mtx;
public:
    static void transfer(BankAccount& from, BankAccount& to, double amount) {
        if (&from < &to) {
            std::lock_guard<std::mutex> l1(from.mtx);
            std::lock_guard<std::mutex> l2(to.mtx);
        } else {
            std::lock_guard<std::mutex> l1(to.mtx);
            std::lock_guard<std::mutex> l2(from.mtx);
        }
        from.balance -= amount;
        to.balance += amount;
    }
};
```

假设账户 A 有 100 元，账户 B 有 50 元。A 向 B 转 30 元，同时 B 向 A 转 20 元。如果两个转账线程各先锁自己的账户再锁对方，就会死锁。按地址排序后，两个线程都以相同顺序加锁，不会循环等待。最终 $A = 100 - 30 + 20 = 90$ 元， $B = 50 + 30 - 20 = 60$ 元，总额 150 元不变。

4.5 互斥量最佳实践

- 使用 RAII：优先用 std::lock_guard，避免手动 lock/unlock
- 临界区尽量短：最小化锁的持有时间
- 加锁顺序一致：防止死锁
- 封装数据和锁：把互斥锁和它保护的数据放在同一个类中

- 避免嵌套锁：尽量不在持有一个锁的同时去获取另一个锁

5 条件变量：std::condition_variable

当线程需要等待某个条件成立时，最直觉的做法是轮询（polling）：循环检查条件，每次检查之间 sleep 一小段时间。但这样做浪费 CPU 周期且引入延迟。条件变量（Condition Variable）提供了更高效的方案：让线程在条件不满足时阻塞等待，直到另一个线程通知它条件可能已满足。

5.1 轮询的问题

```
bool data_ready = false;
std::mutex data_mutex;

void consumer() {
    while (true) {
        {
            std::lock_guard<std::mutex> lock(data_mutex);
            if (data_ready) {
                data_ready = false;
                break;
            }
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        // 浪费 CPU 且引入延迟
    }
}
```

直觉 轮询就像你每隔 5 分钟打一次电话问快递到了没有，既费电话费（CPU 周期），又可能错过刚到的快递（延迟）。条件变量就像让快递员到了直接敲门通知你，你只需安心等待。

5.2 wait 与 notify

条件变量让线程高效地等待某个条件成立。核心 API：

- wait(lock, predicate)：释放锁并阻塞，直到被通知且 predicate 为 true
- notify_one()：唤醒一个等待线程
- notify_all()：唤醒所有等待线程

```
std::mutex m;
std::condition_variable cv;
bool data_ready = false;

void producer() {
    {
        std::lock_guard<std::mutex> lock(m);
        data_ready = true;           // 先修改共享状态
    }
    cv.notify_one();                // 再通知等待的线程
}
```

```
}
```

```
void consumer() {  
    std::unique_lock<std::mutex> lock(m);  
    cv.wait(lock, []{ return data_ready; });    // 等待条件成立  
    // data_ready 为 true, 可以安全处理数据  
}
```

逐行解读：生产者先在锁保护下设置 `data_ready = true`，然后释放锁，再调用 `notify_one` 唤醒消费者。消费者用 `unique_lock` 加锁后调用 `wait`，`wait` 内部会释放锁并阻塞，被唤醒后重新获取锁并检查谓词。

5.3 虚假唤醒

虚假唤醒（Spurious Wakeup）是操作系统的已知行为：`wait` 可能在没有 `notify` 的情况下返回。因此 `wait` 必须带谓词（lambda）：

```
// 推荐写法：带谓词，自动处理虚假唤醒  
cv.wait(lock, []{ return data_ready; });  
  
// 等价于以下循环：  
while (!data_ready) {  
    cv.wait(lock);    // 可能虚假唤醒，所以需要循环检查  
}
```

如果使用不带谓词的 `wait(lock)` 版本，必须自己用 `while` 循环包裹，否则虚假唤醒会导致逻辑错误。推荐始终使用带谓词的版本。

关键细节：先修改共享状态，再 `notify`。如果先 `notify` 再修改状态，消费者可能在状态变更前被唤醒，检查条件发现不满足又继续等待，而生产者不会再 `notify`，导致消费者永久阻塞。

5.4 生产者-消费者模式

生产者-消费者（Producer-Consumer）模式是条件变量最经典的应用：生产者往队列放数据，消费者从队列取数据，队列空时消费者等待。

```
#include <queue>  
#include <mutex>  
#include <condition_variable>  
  
std::queue<int> data_queue;  
std::mutex mtx;  
std::condition_variable cv;  
bool done = false;  
  
void producer() {  
    for (int i = 0; i < 100; ++i) {  
        {  
            std::lock_guard<std::mutex> lock(mtx);  
            data_queue.push(i);    // 生产数据  
        }  
    }  
}
```

```

        cv.notify_one();                // 通知一个消费者
    }
    {
        std::lock_guard<std::mutex> lock(mtx);
        done = true;                    // 标记生产完成
    }
    cv.notify_one();                    // 最后通知一次
}

void consumer() {
    while (true) {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [] { return !data_queue.empty() || done; });
        if (data_queue.empty() && done) break;    // 队列空且生产完成
        int val = data_queue.front();
        data_queue.pop();
        lock.unlock();                      // 处理数据时不需要锁
        // 处理 val...
    }
}

```

逐行解读：生产者循环放入 0 到 99，每次放入后通知一个消费者；全部放入后设置 `done = true` 并最后通知一次。消费者在 `wait` 中等待“队列非空或生产完成”，被唤醒后检查是取数据还是退出。`lock.unlock()` 在处理数据前释放锁，让其他消费者也能取数据，提高并行度。

5.5 线程安全队列

把生产者-消费者模式封装成通用的线程安全队列（Thread-Safe Queue）：

```

template<typename T>
class ThreadSafeQueue {
private:
    std::queue<T> queue_;
    mutable std::mutex mutex_;
    std::condition_variable condition_;
public:
    void push(const T& item) {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            queue_.push(item);
        }
        condition_.notify_one();    // 唤醒一个等待的消费者
    }

    T pop() {
        std::unique_lock<std::mutex> lock(mutex_);
        condition_.wait(lock, [this] { return !queue_.empty(); });
        T item = std::move(queue_.front());
        queue_.pop();
        return item;
    }
}

```



```

    bool empty() const {
        std::lock_guard<std::mutex> lock(mutex_);
        return queue_.empty();
    }
};

```

逐行解读：push 在锁保护下将数据入队，然后释放锁并通知一个等待线程；pop 用 unique_lock 加锁后调用 wait 等待队列非空，被唤醒后用 std::move 取出队首元素避免拷贝。mutable 让 empty() 等 const 方法也能加锁。

例 使用线程安全队列实现一个生产者、两个消费者的小例子：

```

ThreadSafeQueue<int> queue;

void producer(int start, int count) {
    for (int i = start; i < start + count; ++i) {
        queue.push(i);
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}

void consumer(int id) {
    for (int i = 0; i < 5; ++i) {
        int value = queue.pop(); // 队列空时自动阻塞等待
        std::cout << "Consumer " << id << " got: " << value << std::endl;
    }
}

int main() {
    std::thread prod1(producer, 0, 10);
    std::thread cons1(consumer, 1);
    std::thread cons2(consumer, 2);
    prod1.join(); cons1.join(); cons2.join();
    return 0;
}

```

生产者依次放入 0, 1, 2, ..., 9 (共 10 个)，两个消费者各取 5 个。可能的输出：

```

Consumer 1 got: 0
Consumer 2 got: 1
Consumer 1 got: 2
Consumer 2 got: 3
Consumer 1 got: 4
...

```

具体哪个消费者取到哪个值取决于调度，但每个值只会被消费一次，不会丢失也不会重复。10 个数的和为 $0 + 1 + \dots + 9 = 45$ ，两个消费者取到的值之和也一定等于 45，这就是条件变量保证的同步语义。

6 C++17 并行算法

C++17 在标准库中引入了并行版本的 STL 算法，通过添加一个**执行策略**（Execution Policy）参数来控制并行行为。这意味着你可以用一行代码将已有的串行算法变为并行版本。

6.1 执行策略

四种执行策略：

- `std::execution::seq`：顺序执行，等价于传统串行
- `std::execution::par`：并行执行，允许多线程
- `std::execution::par_unseq`：并行且向量化，允许 SIMD
- `std::execution::unseq`：仅向量化（C++20）

```
#include <execution>
#include <algorithm>
```

```
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};
```

```
std::sort(v.begin(), v.end()); // 串行 (C++11)
std::sort(std::execution::par, v.begin(), v.end()); // 并行 (C++17)
```

只需在已有算法调用前加上 `std::execution::par` 参数即可。这与 OpenMP 的 `#pragma omp parallel for` 类似，但更加类型安全且集成在标准库中。

执行策略是一种**许可**（*permission*）而非**保证**（*guarantee*）：标准允许实现选择顺序执行。同时，使用 `par` 时被调用的函数必须是线程安全的，避免共享可变状态。

6.2 常用并行算法

```
// 并行排序
```

```
std::sort(std::execution::par, v.begin(), v.end());
```

```
// 并行遍历：每个元素乘以 2
```

```
std::for_each(std::execution::par, v.begin(), v.end(),
    [](int& x) { x *= 2; });
```

```
// 并行归约求和
```

```
auto sum = std::reduce(std::execution::par, v.begin(), v.end(), 0);
```

```
// 并行变换：平方后存入 output
```

```
std::vector<int> output(v.size());
std::transform(std::execution::par,
    v.begin(), v.end(), output.begin(),
    [](int x) { return x * x; });
```

```
// 变换归约：先平方再求和
```

```
auto squared_sum = std::transform_reduce(
    std::execution::par,
    v.begin(), v.end(),
    0,
```

```
std::plus<>(),
[] (int x) { return x * x; }
);
```

`std::reduce` 是 `std::accumulate` 的并行版本：`accumulate` 要求操作严格从左到右执行，无法并行；`reduce` 允许操作以任意顺序执行（要求操作满足交换律和结合律），因此可以并行化。

6.3 性能考量

并行算法并非总是更快。线程创建和同步有开销，数据量小时串行可能更快：

- 大数据集（通常超过 10000 个元素）适合并行
- CPU 密集型操作适合并行
- 各元素间相互独立的计算适合并行

```
// 小数据集：串行可能更快
std::vector<int> small_data(100);
std::sort(std::execution::seq, small_data.begin(), small_data.end());

// 大数据集：并行有益
std::vector<int> large_data(1000000);
std::sort(std::execution::par, large_data.begin(), large_data.end());
```

7 实战：parallel_accumulate 并行求和

手动实现并行求和，展示了分治并行（Divide-and-Conquer）的典型模式：将数据分成多块，每个线程独立求和，最后汇总各块结果。

```
template<typename Iterator, typename T>
T parallel_accumulate(Iterator first, Iterator last, T init) {
    unsigned long length = std::distance(first, last);
    if (!length) return init;

    // 配置：计算最优线程数
    unsigned long min_per_thread = 25;
    unsigned long max_threads =
        (length + min_per_thread - 1) / min_per_thread;
    unsigned long hardware_threads =
        std::thread::hardware_concurrency();
    unsigned long num_threads =
        std::min(hardware_threads != 0 ? hardware_threads : 2,
                max_threads);
    unsigned long block_size = length / num_threads;

    // 执行：创建线程，每个处理一个数据块
    std::vector<T> results(num_threads);
    std::vector<std::thread> threads(num_threads - 1);

    Iterator block_start = first;
    for (unsigned long i = 0; i < num_threads - 1; ++i) {
        Iterator block_end = block_start;
```

```

    std::advance(block_end, block_size);
    threads[i] = std::thread(
        [&results, i](Iterator f, Iterator l) {
            results[i] = std::accumulate(f, l, T());
        },
        block_start, block_end);
    block_start = block_end;
}
// 主线程处理最后一块
results[num_threads - 1] =
    std::accumulate(block_start, last, T());

// 汇总：等待所有线程，合并结果
for (auto& t : threads) t.join();
return std::accumulate(results.begin(), results.end(), init);
}

```

逐行解读：

- `std::distance(first, last)` 计算数据总长度
- `min_per_thread = 25` 设定每个线程至少处理 25 个元素，避免线程过多导致开销大于收益
- `max_threads` 计算最多需要多少线程（向上取整除法）
- `hardware_concurrency()` 获取硬件支持的线程数，为 0 时回退到 2
- `num_threads` 取硬件线程数和最大线程数的较小值
- `block_size = length / num_threads` 计算每个线程的数据块大小
- 循环创建 `num_threads - 1` 个线程（主线程处理最后一块），每个线程调用 `std::accumulate` 对自己的数据块求和
- Lambda 捕获 `results` 的引用和索引 `i`，将结果存入对应位置
- 最后 `join` 所有线程，用 `std::accumulate` 合并各块结果

例 用小数据验证 `parallel_accumulate` 的正确性。假设数据为 [1, 2, 3, 4, 5, 6, 7, 8], `init = 0`:

1. `length = 8`, `min_per_thread = 25`
2. `max_threads = (8 + 24) / 25 = 1`（数据太少，只需 1 个线程）
3. `num_threads = min(hardware_threads, 1) = 1`
4. 主线程直接处理所有数据： $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 = 36$
如果数据量足够大（如 1000 个元素），`hardware_concurrency()` 返回 8：
1. `max_threads = (1000 + 24) / 25 = 40`
2. `num_threads = min(8, 40) = 8`
3. `block_size = 1000 / 8 = 125`
4. 7 个工作线程各处理 125 个元素，主线程处理最后 125 个
5. 8 个部分和相加得到最终结果

对于 1 到 1000 的连续整数，和为 $(1 + 1000) \times 1000 / 2 = 500500$ ，分块并行求和的结果也一定等于 500500，验证了正确性。

8 框架选型：OpenMP vs C++ Threads vs MPI

并行编程有多种框架，各有适用场景。选择合适的框架是高效开发的关键。

特性	OpenMP	C++ Threads	MPI
学习曲线	低	中	高
开发速度	快	中	慢
细粒度控制	有限	高	高
内存模型	共享	共享	分布式
可扩展性	节点级	节点级	集群级
调试难度	易	中	难

选择建议：

- **OpenMP**：快速并行化已有代码，循环级并行，原型开发
- **C++ Threads**：复杂同步需求，面向对象并发设计，与现代 C++ 特性集成
- **MPI**：跨节点分布式计算，大规模科学计算，异构环境

9 本章你将学会

1. 理解 C++ 并行编程的动机，掌握 RAII、移动语义、Lambda、智能指针等并发前置知识
2. 使用 `std::thread` 创建和管理线程：join/detach、参数传递、RAII 守卫、移动语义、线程标识
3. 识别和解决竞态条件与死锁：mutex/lock_guard/unique_lock 的使用，三种死锁破解方法
4. 使用条件变量实现线程间通信：wait/notify、虚假唤醒防护、生产者-消费者模式与线程安全队列
5. 应用 C++17 并行算法与执行策略，手动实现 `parallel_accumulate` 分治并行求和

10 要点速查

要点	说明
RAII	构造获取，析构释放，异常安全的资源管理
join/detach	线程销毁前必须调用其一，否则 terminate
lock_guard	最常用 RAII 锁，构造加锁析构解锁
unique_lock	灵活锁，支持 defer_lock，条件变量必需
std::lock()	原子锁定多个互斥锁，避免死锁
竞态条件	counter++ 非原子操作，需加锁或用 atomic
死锁	互相等待对方锁，统一顺序 / std::lock / 按地址破解
condition_variable	高效等待条件，wait 必须带谓词防虚假唤醒
Move 语义	thread 不可拷贝只能 move，可放入 vector
Most Vexing Parse	用花括号 {} 统一初始化避免歧义
hardware_concurrency()	查询硬件并发线程数，可能返回 0
C++17 执行策略	seq / par / par_unseq / unseq，许可是非保证
std::reduce	accumulate 的并行版，要求操作满足交换律和结合律

11 小结

本章从“为什么需要 C++ 并行编程”出发，首先快速回顾了面向对象、RAII、移动语义、Lambda、智能指针、模板和 STL 等 C++ 语言基础，为理解并发代码做好铺垫。然后深入讲解了 C++11 并发库的三大支柱：std::thread 的创建与管理（join/detach、参数传递、RAII 守卫、移动语义、线程标识）、std::mutex 的互斥同步（竞态条件、lock_guard/unique_lock、死锁三种破解方法）、std::condition_variable 的条件等待（轮询对比、wait/notify、虚假唤醒、生产者-消费者与线程安全队列）。接着介绍了 C++17 并行算法与执行策略，并通过 parallel_accumulate 展示了分治并行的实战模式。最后对比了 OpenMP、C++ Threads 和 MPI 三种框架的适用场景。

掌握这些概念后，你已经具备了编写正确且高效的 C++ 多线程程序的基础。后续章节将深入 OpenMP、MPI 和 CUDA 等更专业的并行技术。

讲义基于 HPC101-2025 Day7「C++ Concurrency in Action」课程内容编写